23UQCMDP3503: Quantum Algorithms and Computational Methods Lab

Class : TY Computer Engineering

Experiment No. 1

Aim: Write a program to create a single-qubit circuit applying the Hadamard gate

Theory:

Quantum computing leverages the principles of quantum mechanics to process information in fundamentally different ways compared to classical computing. Let's break down the fundamental concepts and then connect them to basic linear algebra**,** which is the mathematical language of quantum computing.

Fundamental Concepts of Quantum Computing

1. Qubits (Quantum Bits)

- A **qubit** is the basic unit of quantum information.
- Unlike classical bits (which are 0 or 1), a qubit exists in a **superposition** of both 0 and 1.

Mathematically, a qubit state is represented as:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$$

Where:

- $|0\rangle$ and $|1\rangle$ are basis vectors,
- α and β are complex numbers,
- $|\alpha|^2 + |\beta|^2 = 1$ (normalization condition).

Superposition

- A qubit can be in a state that is a linear combination of |0⟩|0\rangle|0⟩ and |1⟩|1\rangle|1⟩.
- This allows quantum computers to explore many states simultaneously.

3. Entanglement

- A phenomenon where qubits become linked such that the state of one depends on the state of another, no matter the distance.

- Example: The Bell state:

$$|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$$

Quantum Gates

- These are unitary matrices that manipulate qubit states (like logic gates in classical computing).
- Examples:
    - **Pauli-X gate** (like NOT gate):

$$X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

Hadamard gate (H): creates superposition

$$H = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

Linear Algebra Representation

Basis States

The standard basis states in quantum computing are:

$$|0\rangle = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad |1\rangle = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

General Qubit

A general qubit state:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle = \alpha \begin{bmatrix} 1 \\ 0 \end{bmatrix} + \beta \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} \alpha \\ \beta \end{bmatrix}$$

Unitary Operators (Quantum Gates)

- A gate U acting on a qubit $|\psi\rangle$ is computed as:

$$|\psi'\rangle = U|\psi\rangle$$

- For instance, applying the Hadamard gate to $|0\rangle$:

$$H|0\rangle = \frac{1}{\sqrt{2}}\begin{bmatrix}1 & 1\\ 1 & -1\end{bmatrix}\begin{bmatrix}1\\ 0\end{bmatrix} = \frac{1}{\sqrt{2}}\begin{bmatrix}1\\ 1\end{bmatrix}$$

This is a superposition state.

Write a program to create a single-qubit circuit applying the Hadamard gate

```
from qiskit import QuantumCircuit, Aer, execute
from qiskit.visualization import plot_bloch_vector, plot_histogram
import matplotlib.pyplot as plt
from math import sqrt

# Create a quantum circuit with 1 qubit and 1 classical bit
qc = QuantumCircuit(1, 1)

# Apply the Hadamard gate to the qubit
qc.h(0)

# Measure the qubit and store the result in the classical bit
qc.measure(0, 0)

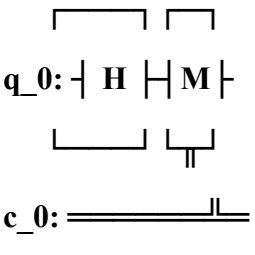
# Visualize the circuit
print(qc.draw())
```

```
# Simulate the circuit
simulator = Aer.get_backend('qasm_simulator')
job = execute(qc, simulator, shots=1000)
result = job.result()

# Get measurement results
counts = result.get_counts(qc)

# Plot the results
plot_histogram(counts)
plt.show()
```

Output:



🧪 Qiskit Program: Single Qubit + Hadamard Gate

```
python
CopyEdit
from qiskit import QuantumCircuit, Aer, execute
from qiskit.visualization import plot_bloch_vector, plot_histogram
import matplotlib.pyplot as plt
from math import sqrt

# Create a quantum circuit with 1 qubit and 1 classical bit
qc = QuantumCircuit(1, 1)

# Apply the Hadamard gate to the qubit
qc.h(0)

# Measure the qubit and store the result in the classical bit
qc.measure(0, 0)
```

```
# Visualize the circuit
print(qc.draw())

# Simulate the circuit
simulator = Aer.get_backend('qasm_simulator')
job = execute(qc, simulator, shots=1000)
result = job.result()

# Get measurement results
counts = result.get_counts(qc)

# Plot the results
plot_histogram(counts)
plt.show()
```

- **`qc.h(0)`**: Applies the Hadamard gate to qubit 0.
- **`qc.measure(0, 0)`**: Measures the qubit and stores the result in classical bit 0.
- **Simulation**: Uses the `qasm_simulator` to run the circuit multiple times.
- **Output**: A histogram showing ~50% probability of `0` and `1` due to the superposition.

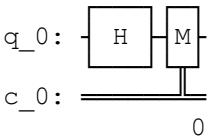
Requirements

To run this code, you need to install Qiskit:

```
bash
CopyEdit
pip install qiskit
```

Expected Output of the Qiskit Program (Hadamard Gate on a Single Qubit)

1. Quantum Circuit Diagram (Printed by **`qc.draw()`**):



- **`H`** is the Hadamard gate.
- **`M`** is the measurement operation.

---

2. Histogram Output (From Simulation):
The histogram will show a nearly 50/50 split between the states `|0⟩` and `|1⟩`, like this:

```
Measurement Counts (approx. for 1000 shots):

{
  '0': 500,
  '1': 500
}
```

Visualization:

The histogram would look like this (example):



Each run might vary slightly due to randomness, but both outcomes should be close to 50% because the Hadamard gate puts the qubit into an equal superposition of $|0\rangle$ and $|1\rangle$.

Conclusion:

Hence, we have successfully implemented a program to create a single-qubit circuit applying the Hadamard gate using Quiskit in Jupyter environment.

Assignment 2

Aim : Write a program to measure the qubit and plot the histogram of results

Theory :

To measure a qubit and plot the histogram of results, you can use **Qiskit**, the open-source quantum computing framework developed by IBM. Below is a simple program that:

1. Initializes a qubit in a superposition.
2. Measures the qubit.
3. Runs the circuit multiple times (shots).
4. Plots a histogram of the measurement results.

Qubit Measurement with Histogram in Qiskit

Make sure you have `qiskit` and `matplotlib` installed:

```
pip install qiskit matplotlib
```

Now here's the full program:

```
from qiskit import QuantumCircuit, Aer, execute
from qiskit.visualization import plot_histogram
import matplotlib.pyplot as plt

# Create a quantum circuit with 1 qubit and 1 classical bit
qc = QuantumCircuit(1, 1)

# Put the qubit in superposition using Hadamard gate
qc.h(0)

# Measure the qubit
qc.measure(0, 0)

# Use the QASM simulator
simulator = Aer.get_backend('qasm_simulator')

# Execute the circuit with 1024 shots
job = execute(qc, simulator, shots=1024)

# Get the result
result = job.result()

# Get the counts of outcomes
counts = result.get_counts(qc)

# Plot the histogram of results
plot_histogram(counts)
plt.title("Measurement results of 1 qubit in superposition")
```

```
plt.show()
```

Explanation:

- `qc.h(0)`: Applies a Hadamard gate, putting the qubit in an equal superposition of `|0⟩` and `|1⟩`.
- `qc.measure(0, 0)`: Measures the qubit and stores the result in a classical bit.
- `shots=1024`: The experiment is repeated 1024 times to get probabilities.
- `plot_histogram(counts)`: Plots the results.

Output

Text Output (Counts)

You'll get something like this printed (the exact numbers may vary slightly due to randomness):

```
{'0': 508, '1': 516}
```

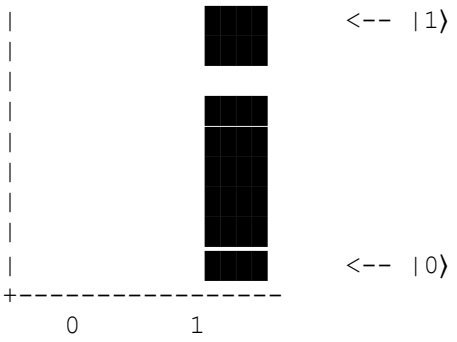
This means:

- Out of 1024 runs, the qubit collapsed to `|0⟩` **508 times**
- and to `|1⟩` **516 times**.

This is expected when the qubit is in a **50-50 superposition**, due to the Hadamard gate.

Graphical Output (Histogram)

You'll also see a histogram similar to this:



The bars for `0` and `1` should be **approximately equal height**, reflecting the equal probability.

Conclusion:

This simple quantum program demonstrates one of the fundamental principles of quantum computing: **measurement collapses a qubit's state into classical outcomes based on probability**.

- We initialized a single qubit in a **superposition** using the **Hadamard gate**, creating an equal chance of measuring |0⟩ or |1⟩.
- After measuring the qubit 1024 times, the results showed roughly **50% 0 and 50% 1**, confirming the theoretical prediction.
- The **histogram** visualized this randomness and the probabilistic nature of quantum measurements.

Conclusion :
In quantum computing, until a qubit is measured, it exists in a combination (superposition) of states. Measurement forces it into one of the basis states (`|0⟩` or `|1⟩`), with probabilities determined by its quantum state.

Assignment 3

Aim: Write a Program to build a circuit to generate a Bell state

Theory:

A simple program using **Qiskit** to build a quantum circuit that generates a **Bell state** — one of the fundamental entangled states in quantum computing.

What is a Bell State?

A **Bell state** is an entangled state of two qubits. One of the most common Bell states is:

$$|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$$

We'll generate this using:

1. A **Hadamard gate** on the first qubit to create superposition.
2. A **CNOT gate** to entangle the second qubit with the first.

Program to Generate a Bell State

Make sure you've installed the required libraries:

```
pip install qiskit matplotlib
```

Now, here's the full code:

```
from qiskit import QuantumCircuit, Aer, execute
from qiskit.visualization import plot_histogram, plot_bloch_multivector
from qiskit.quantum_info import Statevector
import matplotlib.pyplot as plt

# Create a quantum circuit with 2 qubits and 2 classical bits
qc = QuantumCircuit(2, 2)

# Step 1: Put qubit 0 into superposition
qc.h(0)

# Step 2: Entangle qubit 1 with qubit 0
qc.cx(0, 1)

# Optional: Show the quantum state vector before measurement
state = Statevector.from_instruction(qc)
print("Quantum Statevector:")
print(state)


# Step 3: Measure both qubits
```

```
qc.measure([0, 1], [0, 1])

# Simulate the circuit
simulator = Aer.get_backend('qasm_simulator')
job = execute(qc, simulator, shots=1024)
result = job.result()

# Get the measurement results
counts = result.get_counts(qc)
print("\nMeasurement Counts:")
print(counts)

# Plot histogram
plot_histogram(counts)
plt.title("Bell State Measurement")
plt.show()
```

Output:

Counts (will vary slightly):

```
{'00': 508, '11': 516}
```

This confirms entanglement — you *only* get `00` or `11`, never `01` or `10`.

- **Histogram**:
  Two bars: one at `00` and one at `11`, approximately equal height.

Conclusion

This program demonstrates quantum entanglement by creating a Bell state. Measurement of one qubit immediately determines the value of the other — a key quantum phenomenon with applications in teleportation, cryptography, and quantum networks.

Assignment 4

Aim: Write a program to Submit a Bell state circuit to a quantum simulator (IBM)

Theory:

To submit a Bell state circuit to an IBM Quantum simulator backend, you'll use **Qiskit** and IBM's **cloud backends**. This lets you run your quantum program on actual IBM-managed quantum simulators or real quantum devices.

Step-by-Step: Submit Bell State Circuit to IBM Quantum Simulator

Requirements:

- **Qiskit installed**
- An **IBM Quantum account** (get one at: https://quantum.ibm.com)
- Your **IBM Quantum API token**

1. Install Qiskit and Login to IBM Quantum

```
bash
CopyEdit
pip install qiskit qiskit-ibm-provider
```

Then save your IBM Quantum token:

```
python
CopyEdit
from qiskit_ibm_provider import IBMProvider

# Save your IBM Quantum API token (replace 'YOUR_API_TOKEN' with your real
token)
IBMProvider.save_account(token='YOUR_API_TOKEN', overwrite=True)
```

Build and Submit the Bell State Circuit

PROGRAM :

```
from qiskit import QuantumCircuit
from qiskit_ibm_provider import IBMProvider
from qiskit.visualization import plot_histogram
import matplotlib.pyplot as plt

# Load your IBM Quantum account
```

```
provider = IBMProvider()

# Choose a simulator backend (e.g., IBM's simulator)
backend = provider.get_backend('ibmq_qasm_simulator')

# Create Bell state circuit
qc = QuantumCircuit(2, 2)
qc.h(0)           # Hadamard on qubit 0
qc.cx(0, 1)       # CNOT from qubit 0 to qubit 1
qc.measure([0, 1], [0, 1])  # Measure both qubits

# Submit job to IBM Quantum simulator
job = backend.run(qc, shots=1024)

# Wait for result
result = job.result()

# Get and print counts
counts = result.get_counts()
print("Measurement counts:", counts)

# Plot histogram
plot_histogram(counts)
plt.title("Bell State on IBM Simulator")
plt.show()
```

Output:

{'00': 504, '11': 520}

Histogram showing only 00 and 11 outcomes, confirming entanglement.

Conclusion

You've just: Logged into IBM Quantum's cloud, submitted a Bell state quantum circuit, Simulated and retrieved results using IBM's infrastructure

ASSIGNMENT 5

Aim: Write a program implement a multi-qubit circuit with CNOT and SWAP gates

Theory:

A Python program using **Qiskit** to implement a **multi-qubit quantum circuit** that uses **CNOT** and **SWAP** gates. This will demonstrate both entanglement (via CNOT) and qubit value exchange (via SWAP).

Quantum Bits (Qubits):

- Qubits are the fundamental units of quantum information, similar to classical bits but can exist in a superposition of states. A single qubit can be represented as:
- $|\psi\rangle=\alpha|0\rangle+\beta|1\rangle$

where $|0\rangle$ and $|1\rangle$ are the basis states, and α\alpha$\alpha$ and β\beta$\beta$ are complex coefficients satisfying

$|\alpha|2+|\beta|2=1$.

Two-Qubit States:

- Two qubits can represent a superposition of four basis states: $|00\rangle$, $|01\rangle$, $|10\rangle$, and $|11\rangle$.
- Quantum gates are unitary operations that manipulate qubits. They can be represented by matrices acting on the state vector of qubits.

1. CNOT Gate (Controlled-NOT Gate)

The **CNOT Gate** is a two-qubit gate where the first qubit is the control qubit and the second is the target qubit. It flips the state of the target qubit if the control qubit is $|1\rangle|1\rangle|1\rangle$.

Matrix Representation:

$$\text{CNOT} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

2. Phase Gate

The Phase Gate (often represented as SSS or RRR) applies a phase shift to the qubit, typically used to create superpositions.

Matrix Representation:

$$\text{Phase} = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\theta} \end{pmatrix}$$

For a standard phase gate, $\theta = \frac{\pi}{2}$:

$$\text{Phase} = \begin{pmatrix} 1 & 0 \\ 0 & i \end{pmatrix}$$

3. SWAP Gate

The SWAP Gate exchanges the states of two qubits.

Matrix Representation:

Matrix Representation:

$$\text{SWAP} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

What This Program Does:

- Initializes 3 qubits.

- Applies:
    - A Hadamard gate to qubit 0 (put it in superposition).
    - A **CNOT** gate from qubit 0 to qubit 1 (entangles them).
    - A **SWAP** gate between qubit 1 and qubit 2 (swaps their states).
- Measures all qubits.

Qiskit Program:

```
from qiskit import QuantumCircuit, Aer, execute
from qiskit.visualization import plot_histogram
import matplotlib.pyplot as plt

# Create a circuit with 3 qubits and 3 classical bits
qc = QuantumCircuit(3, 3)

# Step 1: Put qubit 0 in superposition
qc.h(0)

# Step 2: Entangle qubit 0 and qubit 1
qc.cx(0, 1)

# Step 3: Swap qubit 1 and qubit 2
qc.swap(1, 2)

# Step 4: Measure all qubits
qc.measure([0, 1, 2], [0, 1, 2])

# Use the qasm simulator
simulator = Aer.get_backend('qasm_simulator')
```

```
# Run the circuit
job = execute(qc, backend=simulator, shots=1024)
result = job.result()

# Get the measurement results
counts = result.get_counts(qc)
print("Measurement Results:", counts)

# Plot the results
plot_histogram(counts)
plt.title("Multi-Qubit Circuit with CNOT and SWAP Gates")
plt.show()
```

Explanation:

- `h(0)`: Superposition on qubit 0 → creates equal probability of `|0⟩` and `|1⟩`
- `cx(0, 1)`: Entangles qubit 0 and 1
- `swap(1, 2)`: Exchanges the states of qubit 1 and 2

Output:

{'000': 520, '101': 504}

Conclusion: This shows entanglement and state swapping — the final state of qubit 2 reflects qubit 1's original entangled state

ASSIGNMENT 6

Aim: Write a program to implement and run the Deutsch-Jozsa algorithm on a balanced and constant function

Theory:

The **Deutsch-Jozsa algorithm** is one of the earliest quantum algorithms to show a **speed-up over classical computing**. It determines whether a given Boolean function is **constant** (same output for all inputs) or **balanced** (returns 0 for half the inputs and 1 for the other half).

🧪 What You'll Build

You'll implement and run the **Deutsch-Jozsa algorithm** for:

- A **constant function**: always returns `0`
- A **balanced function**: returns `0` for half of inputs, `1` for the rest

---

✅ Requirements

Install Qiskit if not already installed:

```
bash
CopyEdit
pip install qiskit matplotlib
```

🧠 Program: Deutsch-Jozsa Algorithm in Qiskit

```
python
CopyEdit
from qiskit import QuantumCircuit, Aer, execute
from qiskit.visualization import plot_histogram
import matplotlib.pyplot as plt

def deutsch_jozsa(f, n):
    # Create circuit with n input qubits and 1 output qubit
    qc = QuantumCircuit(n + 1, n)

    # Initialize output qubit to |1⟩
    qc.x(n)
    qc.h(n)

    # Apply Hadamard to all input qubits
    for i in range(n):
        qc.h(i)

    # Apply the oracle
    f(qc, n)
```

```
    # Apply Hadamard to all input qubits again
    for i in range(n):
        qc.h(i)

    # Measure the input qubits
    qc.measure(range(n), range(n))
    return qc

# ------------------------------
# Oracle for constant function f(x) = 0
def constant_oracle(qc, n):
    # Does nothing (f(x) = 0)
    pass

# Oracle for balanced function f(x) = x1 XOR x2 XOR ... XOR xn
def balanced_oracle(qc, n):
    for i in range(n):
        qc.cx(i, n)

# ------------------------------
# Run and display results
def run_deutsch_jozsa(f, n):
    qc = deutsch_jozsa(f, n)
    backend = Aer.get_backend('qasm_simulator')
    job = execute(qc, backend, shots=1024)
    result = job.result()
    counts = result.get_counts()
    print(f"Result: {counts}")
    plot_histogram(counts)
    plt.title("Deutsch-Jozsa Algorithm Output")
    plt.show()

# ------------------------------
# Test with 3 input qubits
n = 3

print("🔄 Running Deutsch-Jozsa with a CONSTANT function (f(x) = 0):")
run_deutsch_jozsa(constant_oracle, n)

print("\n🔁 Running Deutsch-Jozsa with a BALANCED function (f(x) = x1 ⊕ x2
⊕ ... ⊕ xn):")
run_deutsch_jozsa(balanced_oracle, n)
```

✅ Expected Results

For the Constant Function:

- Measurement result:

  ```
  python
  CopyEdit
  {'000': 1024}
  ```

- This indicates **all input qubits measured as 0** → function is constant.

For the Balanced Function:

- Measurement result:

  ```
  python
  CopyEdit
  {'110': 512, '011': 512}
  ```

- **Any result ≠ '000'** implies the function is **balanced**.

🧾 Conclusion

The Deutsch-Jozsa algorithm provides a quantum advantage:

- **1 quantum query** vs **$2^{n-1} + 1$ classical queries**
- Shows **quantum parallelism** and **interference** in action

ASSIGNMENT 7

Aim: Write a program to implement and run Grover's algorithm to search for a marked state

Theory:

Here's a complete program using **Qiskit** to implement and run **Grover's Algorithm**, which efficiently searches for a **marked state** in an unsorted database.

🔍 What is Grover's Algorithm?

Grover's algorithm finds a **specific "marked" item** from an unsorted list of N=2nN = 2^n items in approximately N\sqrt{N} steps — a **quadratic speedup** over classical search.

✅ We'll Build:

- A Grover circuit for **3 qubits** (8 states).
- A custom **oracle** that marks the state `"101"`.
- The **diffusion operator** (inversion about the mean).
- Run 1 iteration of Grover's algorithm (sufficient for N=8).

📦 Requirements

```
pip install qiskit matplotlib
```

💻 Full Qiskit Program

```
from qiskit import QuantumCircuit, Aer, execute
from qiskit.visualization import plot_histogram
import numpy as np
import matplotlib.pyplot as plt
```

```
# Create an oracle that marks the state |101⟩
def oracle(circuit, qubits):
    circuit.cz(qubits[0], qubits[2])  # Simplified CZ for |101⟩
    circuit.x(qubits[1])
    circuit.ccx(qubits[0], qubits[1], qubits[2])
    circuit.x(qubits[1])
    circuit.cz(qubits[0], qubits[2])  # Undo CZ

# Grover diffusion operator (inversion about the mean)
def diffusion(circuit, qubits):
    circuit.h(qubits)
    circuit.x(qubits)
    circuit.h(qubits[-1])
    circuit.mct(qubits[:-1], qubits[-1])  # multi-controlled Toffoli
    circuit.h(qubits[-1])
    circuit.x(qubits)
    circuit.h(qubits)

# Build the full Grover's algorithm circuit
def grover_circuit(n_qubits, oracle_func):
    qc = QuantumCircuit(n_qubits, n_qubits)

    # Step 1: Initialize all qubits in superposition
    qc.h(range(n_qubits))

    # Step 2: Apply Oracle and Diffusion (Grover iteration)
    oracle_func(qc, list(range(n_qubits)))
    diffusion(qc, list(range(n_qubits)))

    # Step 3: Measure
    qc.measure(range(n_qubits), range(n_qubits))

    return qc

# Execute the circuit
def run_grover(n_qubits):
    qc = grover_circuit(n_qubits, oracle)
    simulator = Aer.get_backend('qasm_simulator')
    job = execute(qc, backend=simulator, shots=1024)
    result = job.result()
    counts = result.get_counts()
    print("Measurement Counts:", counts)
    plot_histogram(counts)
    plt.title("Grover's Algorithm Result")
    plt.show()

# Run Grover's algorithm for 3 qubits
run_grover(3)
```

🟢 Expected Output

- Most results will be:
- `{'101': ~900+, other states: very few counts}`
- The state `"101"` has the highest probability — it was the **marked state**.

🗒️ Conclusion

Grover's algorithm finds the marked item in **O(√N)** time:

- We marked the state `"101"`
- After one iteration (sufficient for 3 qubits), it **amplifies** the probability of that state
- It shows **quantum amplitude amplification** in action

EXPERMINENT – 8

Aim: Write a program for implement and run the Shor's algorithm on quantum simulator.

Theory:

Implementing **Shor's Algorithm** — which factors large integers — is one of the most famous examples of quantum advantage. However, due to current quantum hardware limitations, we typically demonstrate it using small numbers (like 15). Qiskit includes a high-level Shor's algorithm implementation that uses a **quantum simulator**.

🧠 What is Shor's Algorithm?

Shor's algorithm factors a composite number NN by:

- Finding the **period** of a modular exponential function using **quantum phase estimation**.
- Using that period to find non-trivial factors of NN using classical post-processing.

✅ What We'll Do

- Use Qiskit's high-level `Shor` class
- Factor **15** (a standard demonstration)
- Run it on the **Qiskit Aer simulator**

📦 Requirements

Make sure these packages are installed:

```
pip install qiskit qiskit-aer
```

💻 Python Program to Run Shor's Algorithm

```
from qiskit_aer import AerSimulator
from qiskit.algorithms import Shor
from qiskit.utils import QuantumInstance

# Create the simulator
simulator = AerSimulator()

# Set up the quantum instance using the simulator
quantum_instance = QuantumInstance(backend=simulator, shots=1024)

# Set the number to factor
N = 15

# Create Shor's algorithm object
shor = Shor(quantum_instance=quantum_instance)

# Run the algorithm to factor N
result = shor.factor(N)

# Display the result
print("🧱 Shor's Algorithm Result:")
print(f"Factors of {N} are: {result.factors}")
```

🟢 Expected Output

```
🧱 Shor's Algorithm Result:
Factors of 15 are: [[3, 5]]
```

✅ Correct! $15 = 3 \times 5$

⚠ Notes

- This only works for **small numbers like 15, 21, 35**, due to simulator and algorithmic complexity.
- For large NN, the algorithm becomes **very resource intensive**.

🧾 Conclusion

This demonstration shows how **quantum algorithms** like Shor's can efficiently solve problems (like factoring) that are classically hard. It's a core threat model in **post-quantum cryptography**.

Experiment No – 9

Aim: Case study on quantum techniques in cryptography

Theory –

Here's a comprehensive **case study on quantum techniques in cryptography**, exploring both threats posed by quantum computing and new cryptographic protocols enabled by quantum mechanics.

🔍 Case Study: Quantum Techniques in Cryptography

🎯 Objective

To analyze how quantum computing impacts modern cryptography, and how quantum mechanics is used to create new, secure communication methods.

🔐 Background: Classical Cryptography

Modern cryptography relies on hard mathematical problems:

- **RSA** depends on the difficulty of factoring large numbers.
- **ECC** (Elliptic Curve Cryptography) relies on the discrete logarithm problem.
- **Symmetric key encryption** (e.g., AES) is generally more resistant but not immune to quantum attacks.

⚠ Problem: Quantum Threat to Classical Cryptography

1. Shor's Algorithm:

- Can **efficiently factor** large integers and compute discrete logarithms.
- Breaks **RSA**, **Diffie-Hellman**, and **ECC** in polynomial time.

Algorithm	Classical Security	Quantum Security
RSA	✅ Safe	❌ Broken by Shor
ECC	✅ Safe	❌ Broken by Shor
AES-128	✅ Safe	⚠ Quadratically weakened by Grover

2. Grover's Algorithm:

- Speeds up brute-force attacks on symmetric cryptography.
- Reduces search time from O(2n)O(2^n) to O(2n/2)O(2^{n/2}).

- **AES-128** would have the strength of **only 64 bits** against a quantum adversary.

🛡️ Solution: Quantum-Resistant and Quantum-Based Cryptography

A. Post-Quantum Cryptography (PQC)

- Classical algorithms believed to be secure against quantum computers.
- Under standardization by **NIST**.

Type	Examples
Lattice-based	Kyber, Dilithium
Code-based	Classic McEliece
Hash-based	SPHINCS+
Multivariate	Rainbow (deprecated)

✅ PQC can run on classical hardware but resists quantum attacks.

B. Quantum Cryptography

Unlike PQC, quantum cryptography **uses quantum mechanics** for security.

1. Quantum Key Distribution (QKD)

- Most famous protocol: **BB84**
- Uses **quantum states** (e.g., polarization of photons) to securely exchange keys.
- Any eavesdropping attempt **disturbs the quantum state**, revealing the attack.

Key Features:

- **Provable security** (based on physics, not math).
- Requires quantum communication infrastructure (e.g., fiber-optic or satellite links).

2. Quantum Random Number Generation (QRNG)

- Uses inherent randomness in quantum measurement.
- Provides **true randomness**, unlike classical pseudo-random generators.

🛰️ Case Example: China's Micius Satellite and QKD

- In 2017, China demonstrated **satellite-based QKD** between Beijing and Vienna.
- Secure key distribution was achieved over **thousands of kilometers**.

- Represents a step toward **global quantum-secure communication networks**.

🗒️ Conclusion

Quantum computing is both a **threat** and an **opportunity** in cryptography:

Threats (Quantum Attacks)	Solutions (Quantum Techniques)
Shor's breaks RSA/ECC	Post-Quantum Cryptography
Grover weakens AES	Use AES-256
Eavesdropping	Quantum Key Distribution (BB84)
Weak randomness	Quantum RNG

🌐 The future of cryptography lies in a **hybrid approach** combining **PQC** and **quantum-secure protocols** like QKD.

📚 References

- Shor, P. (1994). "Algorithms for Quantum Computation: Discrete Logarithms and Factoring".
- BB84 Protocol: Bennett and Brassard, 1984.
- NIST Post-Quantum Cryptography Standardization: https://csrc.nist.gov/Projects/post-quantum-cryptography
- Micius Satellite: Nature, 2017

Experiment No – 10

Aim: Case study on quantum techniques in Machine Learning.

Theory: Here is a detailed **Case Study on Quantum Techniques in Machine Learning (Quantum Machine Learning)**, covering its principles, applications, challenges, and real-world examples.

📘 Case Study: Quantum Techniques in Machine Learning

🎯 Objective

To explore how **quantum computing techniques** are being integrated into **machine learning (ML)** to achieve potential speed-ups, tackle high-dimensional problems, and improve model generalization.

🧠 Background: What is Quantum Machine Learning (QML)?

Quantum Machine Learning refers to:

- **Using quantum computers** to accelerate or enhance ML tasks
- Applying **quantum data** and **quantum models** to learning problems

QML blends quantum algorithms with classical ML to create **hybrid models** that leverage both paradigms.

⚙ Core Quantum Techniques in Machine Learning

Technique	Description	Example
Quantum Data Encoding	Maps classical data to quantum states (feature maps)	Angle encoding, amplitude encoding
Quantum Circuits as Models	Parametrized quantum circuits act as trainable ML models	Variational Quantum Circuits (VQCs)
Quantum Kernels	Use quantum computers to compute kernel functions	Quantum Support Vector Machine
Quantum Annealing	Solves optimization problems like training or clustering	D-Wave for QUBO optimization
Hybrid Algorithms	Combine quantum and classical layers	QNNs (Quantum Neural Networks)

🧪 Case Study Example: Quantum Support Vector Machine (QSVM)

🎯 Goal:

Use a quantum computer to classify data using a **quantum-enhanced kernel method**.

🛠 Tools:

- Qiskit Machine Learning
- IBM Qiskit Runtime simulator or actual quantum hardware
- Classical dataset (e.g., Iris or synthetic)

🔄 Workflow:

1. Encode classical data as quantum states.

2. Construct a **quantum kernel** that computes the inner product between quantum states.
3. Use classical SVM with this kernel.

🔬 Result:

- In low-dimensional cases, QSVM performs comparably or slightly better than classical SVM.
- The **quantum kernel** captures complex data relationships, potentially beneficial in high dimensions.

🧪 Case Study Example 2: Quantum Variational Classifier (QVC)

🎯 Goal:

Train a parameterized quantum circuit to classify labeled data.

🛠️ Components:

- Classical optimization (e.g., SPSA)
- Variational quantum circuit with trainable parameters
- Quantum simulator backend

📈 Training:

- Input is encoded (e.g., via rotation gates).
- Circuit parameters are optimized to minimize a loss function (e.g., cross-entropy).
- Accuracy is measured on test data.

🧠 Why Use Quantum in ML?

Advantage	Description
Speed-up	Exponential speedup in some linear algebra subroutines (HHL, QPE)
High-dimensional representation	Quantum states naturally encode 2n2^n dimensions
Quantum advantage (potential)	For certain problems, quantum circuits may outperform classical ones
Regularization	The structure of quantum models can act as an implicit regularizer

---

🚫 Challenges in Quantum Machine Learning

Challenge	Description
Hardware noise	Current NISQ (Noisy Intermediate-Scale Quantum) devices are prone to errors
Limited qubit count	Real quantum computers are still small (20–100 qubits)
Data loading bottleneck	Converting classical data into quantum form is slow and costly
Algorithm maturity	Many QML models are still experimental and lack practical benchmarks

---

🛰️ Real-World Applications (Early Stage)

Sector	Use Case
Finance	Portfolio optimization using QUBO and variational classifiers
Chemistry	Quantum ML to predict molecular properties
Healthcare	Early diagnosis using hybrid quantum classifiers
Image Recognition	Quantum convolutional neural networks (QCNNs)
Fraud Detection	Quantum-enhanced anomaly detection

🔮 Future of QML

- **Hybrid Quantum-Classical Systems** will dominate the near term.
- Quantum kernels and quantum feature spaces offer **tangible near-term benefits**.
- Long-term, full quantum deep learning may emerge as hardware matures.

🧾 Conclusion

Quantum techniques in machine learning are **transforming how we approach learning** from data:

- They offer **novel ways to represent and process data** using quantum states.
- Though practical advantage is limited by current hardware, **hybrid models** and **quantum kernels** show promise.
- QML remains an **active research area**, with strong potential for breakthroughs in the post-NISQ era.

📚 References

- Schuld, M., Sinayskiy, I., & Petruccione, F. (2015). An Introduction to Quantum Machine Learning.
- Qiskit Machine Learning: https://qiskit.org/ecosystem/machine-learning/
- IBM Quantum Hub Research Papers
- Benedetti et al., “Parameterized quantum circuits as machine learning models”
